Postgres for pgops — a deep dive

Internals through the DBA workflow, with a psql lab for every concept

June 2026

Contents

Postgres for pgops — a deep dive **1**

Part A — The mental frame **2**
A.1 A day in the life of a Postgres DBA . . . 2
A.2 The back office in one diagram . . . 2
A.3 The stat-view dashboard . . . 4

Part B — The four back-office layers **5**
B.1 The Planner . . . 6
B.2 The Journal (WAL) . . . 8
B.3 MVCC and the Vacuum . . . 10
B.4 The Locks . . . 13

Part C — The lab **16**
11.0 Shared setup . . . 16
11.1 Scenario 1 — on-call shift against the e-commerce DB . . . 16
11.2 Scenario 2 — standalone mechanism-reveal sandboxes . . . 22

Part D — Quick reference **24**

Sources **26**

Postgres for pgops — a deep dive

The shape of this doc. Part A is the mental frame: what a Postgres DBA actually does, and what part of the database they're staring at when they do it. Part B is the four back-office layers behind that frame, each with the workflow that drives it. Part C is the lab — nine `psql` exercises against a single seeded e-commerce DB, tagged with which frame piece each one demonstrates. Part D is the quick-reference card.

Postgres version. PG 18.4 throughout (latest stable as of writing [1]). Behavior differences vs PG 13–17 called out inline.

Part A — The mental frame

A.1 A day in the life of a Postgres DBA

A Postgres DBA does one thing in a loop: **observe** the database, **diagnose** the anomaly, **intervene**. Every alert, every paging incident, every weekly review fits that shape. The toolkit you need is built around it [4].

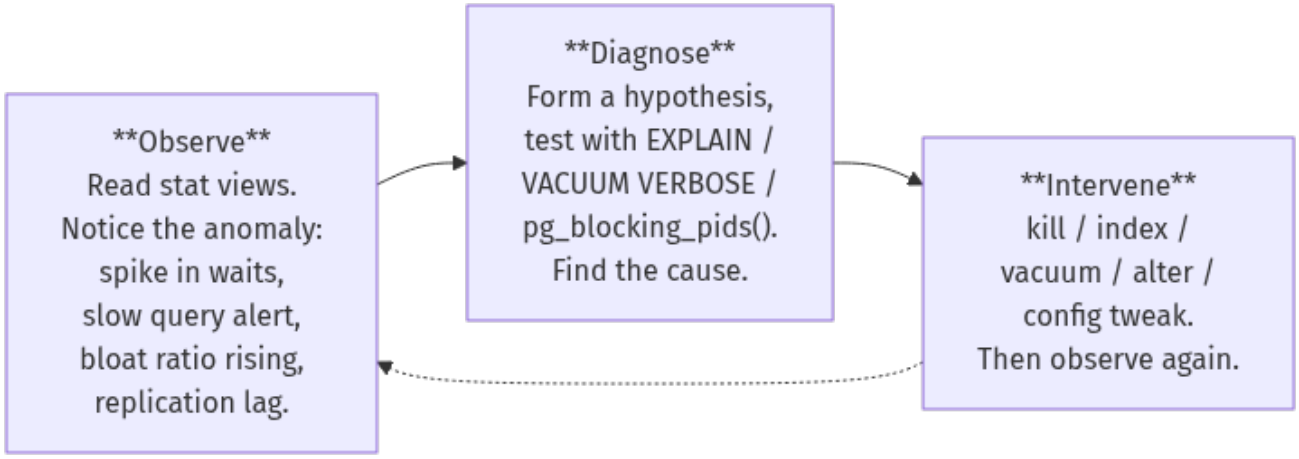


Figure 1: Diagram 1

A worked example: PagerDuty fires for elevated p99. You *observe* `pg_stat_activity` and see 30 sessions in `Lock` waits. You *diagnose* with `pg_blocking_pids()` — root blocker is one session, `idle in transaction` for 14 minutes, holding a `FOR UPDATE` on `orders`. You *intervene* with `pg_terminate_backend()`. Loop closes.

A second worked example (different layer, same loop): the disk-usage graph trends up steadily without a write spike to explain it. You *observe* `pg_replication_slots` and see one slot with `active=false` and 142 GB of `retained_wal`. You *diagnose* by checking `pg_stat_replication` — the named subscriber isn't connected, hasn't been for weeks. You *intervene* with `pg_drop_replication_slot()`; the next checkpoint recycles ~140 GB of WAL.

A third (slow over weeks, not minutes): the morning bloat report shows `orders` at 38% dead tuples. You *observe* `pg_stat_user_tables` and confirm `last_autovacuum` is from two weeks ago. You *diagnose* via `pg_stat_activity WHERE backend_xmin IS NOT NULL` — an analyst's BI tool is leaving REPEATABLE READ snapshots open for hours, pinning the xmin horizon. You *intervene* by setting `idle_in_transaction_session_timeout='5min'` and a one-off `VACUUM (VERBOSE, ANALYZE) orders;` to catch up.

All three are the same loop, but the *trigger view* and the *intervention primitive* differ by layer. Part B is the four layers, in the order they get queried during a typical incident.

Scope of this doc. The full DBA job is install/maintain + performance + backup/recovery + security/RBAC + extensions [8]. This document covers the **performance + on-call** slice — the layers you query and intervene in during an incident. Backups (use pgbackrest or WAL-G), security (pg_hba.conf + GRANT), and extension management live in other docs.

A.2 The back office in one diagram

What's actually happening when you `SELECT`:

The pieces a DBA spends time inside:

- **Backend.** The OS process serving one connection. Every backend has its own `work_mem` budget; backends share `shared_buffers`. Because it's one OS process per session, every serious deployment runs `pgbouncer` to multiplex.
- **Planner.** Chooses how to execute your SQL. The single biggest perf lever, because two queries that look identical can differ 10000× in cost depending on the plan.
- **Executor.** Walks the plan, reads pages from `shared_buffers` (loading from disk if needed), appends writes to WAL.

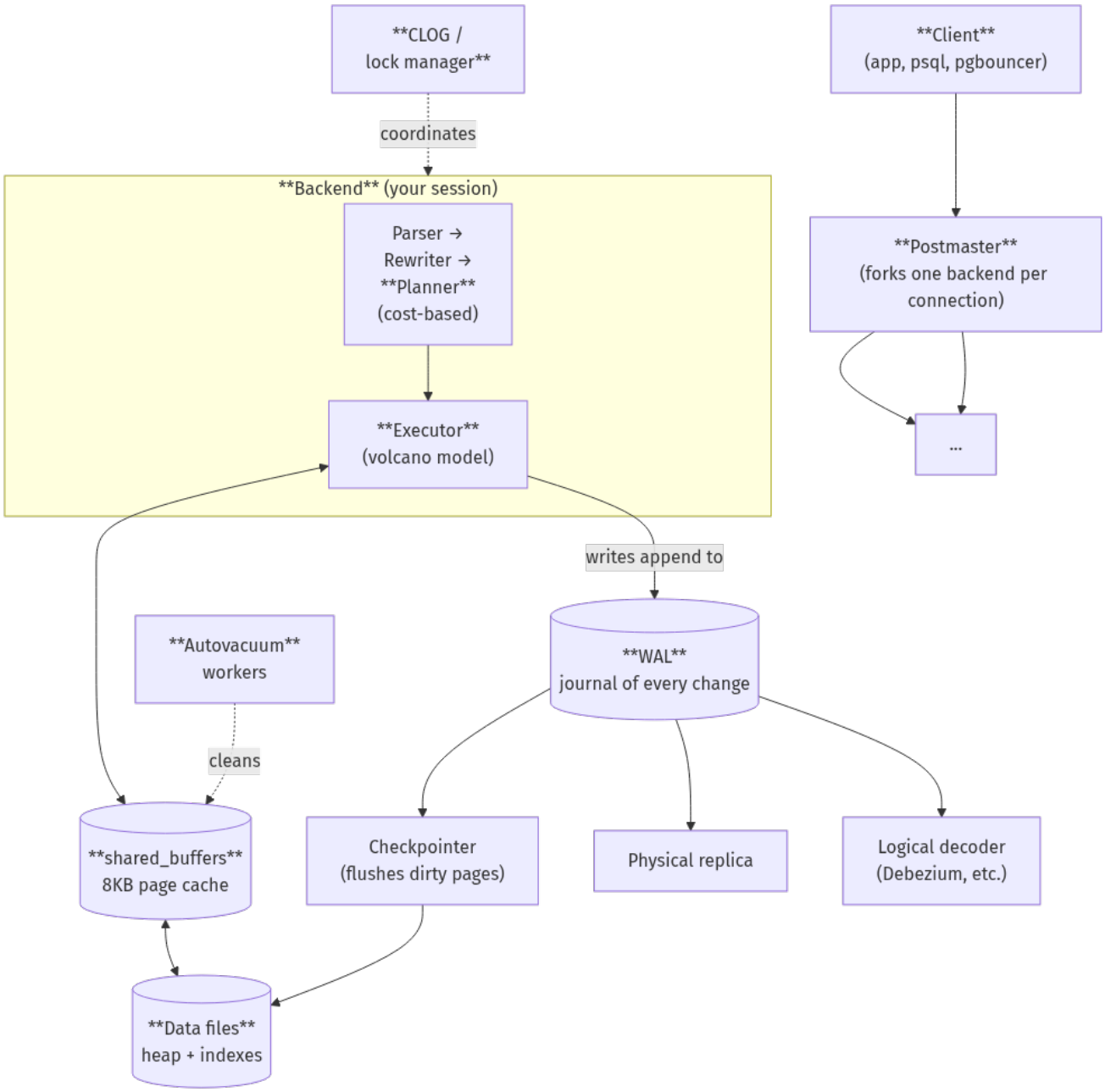


Figure 2: Diagram 2

- **WAL.** Source of truth. Crashes recover by replaying WAL. Replicas exist by streaming WAL. Logical CDC tools decode WAL.
- **Autovacuum.** Cleans up the dead row versions MVCC leaves behind. Tunable per-table.
- **Lock manager.** Coordinates conflicting writers. Mostly invisible until something goes wrong.

A.3 The stat-view dashboard

Six SQL views are the DBA's permanent windows into the back office. Everything else is reference [7].

View	What it reveals	Loop step it serves	Layer (Part B)
`pg_stat_activity`	One row per session: query, state, wait event, txn age, client IP	Observe (constantly), Diagnose	All four
`pg_stat_statements`	Per-normalized-query rollups: calls, total time, mean time, cache hit %	Observe, Diagnose	B.1 Planner
`pg_stat_user_tables`	Per-table: live/dead tuples, last vacuum, vacuum count, HOT ratio	Observe (daily), Diagnose	B.3 MVCC + Vacuum
`pg_locks`	One row per held/waited lock: locktype, mode, granted, relation	Diagnose (incident-time)	B.4 Locks
`pg_replication_slots`	One row per slot: active, restart_lsn, lag, plugin	Observe (alerting)	B.2 WAL
`pg_stat_io` *(PG 16+)*	I/O per backend type × context: reads, hits, writes, evictions	Observe (capacity)	Cross-cutting

Everything that follows in Part B is "how this view's columns are produced inside the database." The shape of the doc is: views first (you'll never stop looking at them), then the machinery behind them.

The queries you actually type all day. These are the dozen-ish `SELECT`s a DBA leaves in a scratch file and copy-pastes during incidents. The full SQL lives in the labs; what you type to reach the answer fits in a paragraph:

- "What is everyone doing right now?" → `SELECT pid, state, wait_event_type, wait_event, query_start, query FROM pg_stat_activity WHERE state != 'idle' ORDER BY query_start;`
- "Who's blocking whom?" → `pg_stat_activity` joined on `pg_blocking_pids()` (full version in lab 11.1.C).
- "What are my worst queries by total time?" → `SELECT substring(query,1,60), calls, total_exec_time FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 20;`
- "Which tables need vacuum?" → `SELECT relname, n_dead_tup, last_autovacuum FROM pg_stat_user_tables ORDER BY n_dead_tup DESC LIMIT 20;`
- "Am I close to wraparound?" → `SELECT relname, age(relfrozenxid) FROM pg_class WHERE relkind='r' ORDER BY age(relfrozenxid) DESC LIMIT 10;`
- "Are my replication slots healthy?" → `SELECT slot_name, active, pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn) AS lag FROM pg_replication_slots;`

Once those six queries are second nature, you've covered ~80% of the daily observation surface.

Part B — The four back-office layers

A pedagogical decomposition focused on the **diagnosable on-call surface**. The four layers cover where DBAs actually intervene during incidents.

Suzuki's *Internals of PostgreSQL* [5] and surveys like "Inside PostgreSQL: MVCC, WAL, TOAST, VACUUM" [6] cover the same underlying machinery but split differently — Suzuki has 12 chapters by subsystem; the survey blogs trace one journey across them. Both shapes work for *teaching* the database. For on-call work, the right cuts are along *what fires the page*: slow query (Planner), replication anomaly (WAL), bloat (MVCC + Vacuum), contention (Locks). That's the cut Part B takes.

What's intentionally cross-cutting and not its own chapter:

- **Buffer cache** (`shared_buffers`) — every layer reads through it. Surfaces in EXPLAIN (BUFFERS), `pg_buffercache`, `pg_stat_io`. Discussed in each layer where it matters.
- **Backups** — use pgbackrest or WAL-G; the ops discipline (RPO/RTO targets, restore-test cadence) is the same; the database internals don't change.
- **Extensions** — install once via `shared_preload_libraries` + `CREATE EXTENSION`; thereafter just SQL.
- **Security / RBAC** — `pg_hba.conf` for auth, `GRANT` for permissions, `pg_monitor` role for read-only DBAs.
- **Connection pooling** — pgbouncer is essential because Postgres is process-per-connection (Part A.2), but it's an external component.

They're not absent from a real DBA's job — they're absent from this doc.

B.1 The Planner

DBA loop step: Diagnose. **Workflow trigger:** APM flags an endpoint slow; or `pg_stat_statements` shows a query burning total time. **Mental model:** Every SELECT/UPDATE has a *plan* — the choice the database made about how to run it. Plans differ by 10000× in cost. The planner picks based on table sizes (statistics in `pg_statistic`) and indexes available. **Query to see it:** `EXPLAIN (ANALYZE, BUFFERS) <stmt>` for one query; `SELECT * FROM pg_stat_statements ORDER BY total_exec_time DESC` for the worst over time.

What the planner knows

The planner reads four inputs:

1. **The query** itself (parsed, rewritten).
2. **The schema** — tables, columns, indexes.
3. **Per-column statistics** in `pg_statistic`: row counts, null fractions, distinct counts, **MCVs (most common values) with frequencies**, **histogram bounds** for range estimates. Populated by `ANALYZE`.
4. **Cost knobs** as GUCs: `seq_page_cost` (1.0), `random_page_cost` (4.0 — set to 1.1 on SSD), `cpu_tuple_cost` (0.01), `cpu_index_tuple_cost` (0.005).

From these it builds **row estimates** and **cost estimates**, enumerates valid plans, picks the lowest cost.

The single most common failure mode is stale statistics. No recent ANALYZE → bad MCVs → planner estimates "1 row matches" when really 1M rows match → picks Nested Loop → query is 1000× slower than it should be. ANALYZE is the cheapest fix in Postgres; run it after every bulk load.

Indexes — the planner's primary lever

Type	Use when	Notes
B-tree	95% of cases — equality, range, sort, prefix	Default. When in doubt, B-tree.
GIN	Full-text on `tsvector`; JSONB `@>` containment; array `@>`	Write-amplified. `fastupdate` batches inserts.
GiST	PostGIS, range `&&` overlap, KNN `<->` ORDER BY	"Contains/overlaps/nearest" framework.
BRIN	Huge naturally-ordered tables (time-series logs)	Tiny — 1000× smaller than B-tree. Coarse but cheap.
Hash	Skip — B-tree beats it everywhere.	Crash-safe since PG 10 but rarely justified.

Variants:

- **Partial** — `CREATE INDEX … WHERE status='active'`. Smaller, faster, but predicate must match.
- **Expression** — `CREATE INDEX … (lower(email))`. Enables `WHERE lower(email) = ?`.
- **Covering** — `CREATE INDEX … (a) INCLUDE (b, c)`. Enables index-only scans.

`CREATE INDEX CONCURRENTLY` is the lock-friendly variant: two table scans, weaker lock (ShareUpdateExclusive), doesn't block writes. The catch: a failed CONCURRENTLY build leaves an `INVALID` index that consumes space and is updated by writes but does nothing useful. Always check:

```
SELECT indexrelid::regclass FROM pg_index WHERE NOT indisvalid;
```

Drop with `DROP INDEX CONCURRENTLY`, retry the build.

Reading EXPLAIN

Plans are trees. Read **bottom-up, inside-out**. Most-indented nodes execute first; parents pull rows from children.

```
 Hash Join  (cost=125.50..1842.30 rows=12042 width=187)
            (actual time=2.110..18.341 rows=12041 loops=1)
   ->  Seq Scan on orders o (rows=12042) (actual rows=12041)
```

```
->  Hash  (rows=4801) (actual rows=4801 time=2.020..2.021)
      ->  Seq Scan on customers c (rows=4801) (actual rows=4801)
```

What matters in the numbers:

- **rows estimated vs rows actual.** 10× discrepancy = the planner is wrong, often the cause of a bad plan. Fix: ANALYZE. If still wrong, look at `CREATE STATISTICS` for correlated columns.
- **cost** is unitless — only useful for *relative* comparison between plans.
- **loops > 1** on a Nested Loop's inner side: actual work = `rows × loops`, not the per-loop count.

Add `(ANALYZE, BUFFERS)` for cache hit/read counts per node — `shared hit=X read=Y` tells you whether the query was CPU-bound (high hit) or I/O-bound (high read).

Join algorithms — picked by the planner

- **Nested Loop** — outer loop, indexed lookup on inner. Wins when outer is small + inner has an index. `O(N log M)`.
- **Hash Join** — build hash from smaller side, probe with larger. Wins on equality joins with no useful index. Needs the hash table to fit `work_mem` — if not, spills to temp files (much slower).
- **Merge Join** — sort both sides, walk in tandem. Wins when both sides already sorted (post-existing indexes), or `work_mem` is too small for Hash.

Parallel and JIT

Parallel query (default on since 9.6): planner adds `Gather` nodes when table size > `min_parallel_table_scan_s` (default 8 MB). Caps: `max_parallel_workers_per_gather` (2), `max_parallel_workers` (8 cluster-wide).

JIT (PG 11+): LLVM-compiles expressions when plan cost > `jit_above_cost` (default 100000). Can hurt analytical queries that plan expensive but execute fast — the compile is overhead. If `EXPLAIN ANALYZE` shows non-trivial JIT timing, try `SET jit = off`.

Views you query for this layer

`pg_stat_statements` is the primary one. After `CREATE EXTENSION pg_stat_statements;` (per database) and `shared_preload_libraries=pg_stat_statements` (cluster-wide, requires restart), every executed query is normalized (constants → placeholders) and tracked. Columns that matter:

- `queryid` — bigint hash of the normalized text. Same logical query → same hash.
- `query` — the normalized text. **Truncated at `pg_stat_statements.track_utility_text_size`** (default 1024 bytes).
- `calls`, `total_exec_time`, `mean_exec_time`, `stddev_exec_time`, `rows`.
- `shared_blks_hit`, `shared_blks_read` — buffer cache hit/miss counts.
- `temp_blks_written` — blocks spilled to temp files (sort/hash exceeded `work_mem`).
- `stats_reset` — last reset timestamp.

Gotchas: capped at `pg_stat_statements.max` entries (default 5000) — LRU eviction when full; `pg_stat_statements_reset()` zeroes everything (use after a deploy to measure fresh); two queries differing only by constants collapse to one entry.

`pg_statio_user_tables` / **`pg_statio_user_indexes`** show block-level cache hit vs disk read per table or index. Useful for "is my hot table actually hot in cache, or always cold-reading?"

`pg_stat_io` (PG 16+) is the modern catch-all. One row per `(backend_type, object, context)`. Backend types include `client backend`, `autovacuum worker`, `checkpointer`. Contexts include `normal`, `vacuum`, `bulkread`, `bulkwrite`. Columns: `reads`, `writes`, `extends`, `hits`, `evictions`, `reuses`. Replaces a lot of "what is shared_buffers actually doing" guesswork.

What you intervene with

`CREATE INDEX CONCURRENTLY`, `ANALYZE`, `CREATE STATISTICS` (for correlated columns where the planner mis-estimates), GUC tweaks (`work_mem`, `random_page_cost` to 1.1 on SSD), occasional query rewrite. See lab 11.1.A (planner tour) and 11.1.B (slow query → index).

B.2 The Journal (WAL)

DBA loop step: Observe (continuous), Intervene (when something pins WAL). **Workflow trigger:** Replication lag alert; disk filling without obvious cause; a logical subscriber stopped consuming. **Mental model:** Every change appends a record to the WAL before any data file is touched. WAL is the source of truth. Replay it after crash → recovery. Stream it → replication. Decode it → CDC. *Disks fill when WAL piles up because something is preventing recycling.* **Query to see it:** `SELECT pg_current_wal_lsn()` for the write head; `SELECT * FROM pg_replication_slots` for who's pinning what.

WAL anatomy

Every INSERT/UPDATE/DELETE/CREATE INDEX/ALTER appends a record to `pg_wal/` as 16 MB segment files. Each record carries a small header (length, prev-record pointer, resource manager id, info flags, CRC32) plus a resource-manager payload (Heap / Btree / Transaction / Standby / Logical / ~20 others).

LSN — Log Sequence Number. A 64-bit byte offset into the WAL stream, printed hex like `0/16B5A88`. Compute bytes between two LSNs with `pg_wal_lsn_diff()`. LSNs only increase within a *timeline* — a failover increments the timeline id.

Checkpoints

A checkpoint flushes all dirty `shared_buffers` pages to data files and writes a checkpoint record to WAL. Crash recovery starts from the most recent checkpoint, so checkpoints bound recovery time.

Triggers:

- **Time** — `checkpoint_timeout` (default 5 min) elapsed.
- **WAL volume** — `max_wal_size` (default 1 GB) written since last checkpoint.
- **Manual** — `CHECKPOINT;`.

Postgres spreads checkpoint I/O across `checkpoint_completion_target × checkpoint_timeout` (default 0.9 × 5 min = 4.5 min) to avoid latency spikes. Bulk-load workloads benefit from raising `max_wal_size` so checkpoints don't fire mid-load.

Replication — physical vs logical

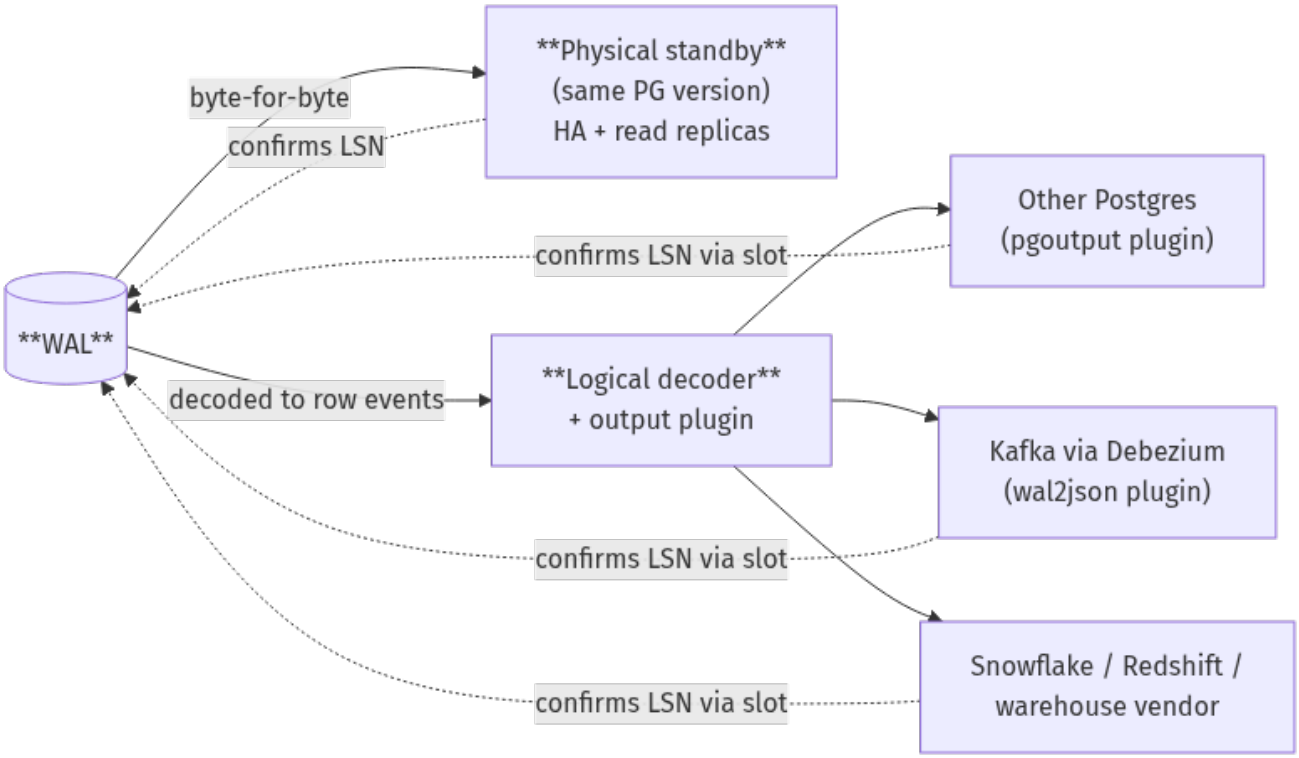


Figure 3: Diagram 3

Physical replication — standby receives WAL bytes and applies them as-is. Same Postgres major version. Used for HA failover, read replicas.

Logical replication — Postgres *decodes* WAL records back into row-level change events (INSERT/UPDATE/DELETE with column values), streams via a plugin. Bundled plugin is `pgoutput` (binary protocol). `wal2json` (text) is used by Debezium and other CDC tools. Requires `wal_level=logical` on the publisher.

Replication slots — the bear trap

A **slot** is the publisher's per-subscriber bookkeeping. It has a `restart_lsn` (oldest WAL the subscriber might need) and `confirmed_flush_lsn` (subscriber's confirmed position).

Postgres will not recycle WAL past `min(restart_lsn)` across all active slots. Without a slot, WAL recycles per the standard rules. With a slot pinning old WAL, WAL accumulates indefinitely. This is the **orphan-slot pattern** — single biggest replication-related outage cause:

1. A subscriber dies (process crash, decommissioned, slot forgotten).
2. Slot remains defined, `restart_lsn` frozen.
3. WAL accumulates byte by byte since.
4. Disk fills. Primary crashes or refuses writes.

Detection: `active=false` AND non-trivial `pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)`. Mitigation: confirm subscriber really gone, then `SELECT pg_drop_replication_slot('slot_name');`. WAL recycles on next checkpoint.

`max_slot_wal_keep_size` (PG 13+) is a hard cap: if a slot's retained WAL exceeds it, the slot is invalidated (`wal_status='lost'`), the subscriber permanently loses its position, but the disk doesn't fill. Set this to something you can afford to lose.

Views you query for this layer

`pg_replication_slots` is the primary one. Columns:

- `slot_name`, `plugin` (null for physical slots; `pgoutput` / `wal2json` / `test_decoding` etc. for logical), `slot_type`, `database` (null for physical).
- `active` + `active_pid` — is a subscriber connected right now?
- `restart_lsn` — oldest WAL position the slot might still need. **WAL is pinned from here forward.**
- `confirmed_flush_lsn` — subscriber's confirmed durable position.
- `wal_status` (PG 13+) — `reserved` / `extended` / `unreserved` / `lost`. `lost` means `max_slot_wal_keep_size` invalidated the slot — subscriber permanently lost its position.

Useful derived numbers: lag bytes = `pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)`. ETA-to-disk-exhaustion comes from tracking the lag over time and projecting.

`pg_stat_replication` shows every connected physical standby: `application_name`, `client_addr`, `state` (`streaming` / `catchup` / `startup`), `sent_lsn` / `write_lsn` / `flush_lsn` / `replay_lsn`. Replication lag in bytes is `pg_current_wal_lsn() - replay_lsn`.

`pg_ls_waldir()` lists what's actually on disk in `pg_wal/` — segment files with sizes and modification times. Useful sanity check when the disk-usage graph and the slot lag don't agree.

What you intervene with

`pg_drop_replication_slot()` for the orphan. `CHECKPOINT;` to force WAL recycle after dropping. `max_slot_wal_keep_size` GUC as a permanent safety net. For physical replication lag: investigate network or standby health (rarely the primary). See lab 11.1.F (logical decoding + orphan slot).

B.3 MVCC and the Vacuum

DBA loop step: Diagnose. **Workflow trigger:** A table feels bloated; autovacuum can't keep up; wraparound horizon alert. **Mental model:** Rows aren't overwritten — UPDATE writes a new version, marks the old dead. Readers see snapshots. Vacuum cleans the dead versions. *A long-running transaction blocks all cleanup* because it might still want to read those dead rows. **Query to see it:** `SELECT relname, n_dead_tup, last_autovacuum FROM pg_stat_user_tables ORDER BY n_dead_tup DESC;`

The core trick

Every row carries hidden columns:

- **`xmin`** — txn id that created this version.
- **`xmax`** — txn id that invalidated it (set by UPDATE/DELETE), or 0 if still live.

`INSERT` writes a new row, `xmin=current_xid, xmax=0`. `UPDATE` writes a *new* row with `xmin=current_xid` and sets the *old* row's `xmax=current_xid` — the old is now a **dead tuple**, physically present, invisible, waiting for VACUUM. `DELETE` just sets xmax on the existing row.

A **snapshot** is the set of xids you can see: `(snapshot.xmin, snapshot.xmax, xip[])`. A row version is visible if its xmin is committed and < snapshot.xmax and not in xip[], AND its xmax is either 0, aborted, ≥ snapshot.xmax, or in xip[].

Three isolation levels are just snapshot policies:

Level	Snapshot taken at
READ COMMITTED (default)	Start of each statement
REPEATABLE READ	Start of transaction; reused
SERIALIZABLE	REPEATABLE READ + cycle detection via SSI

The lifecycle, with the trap

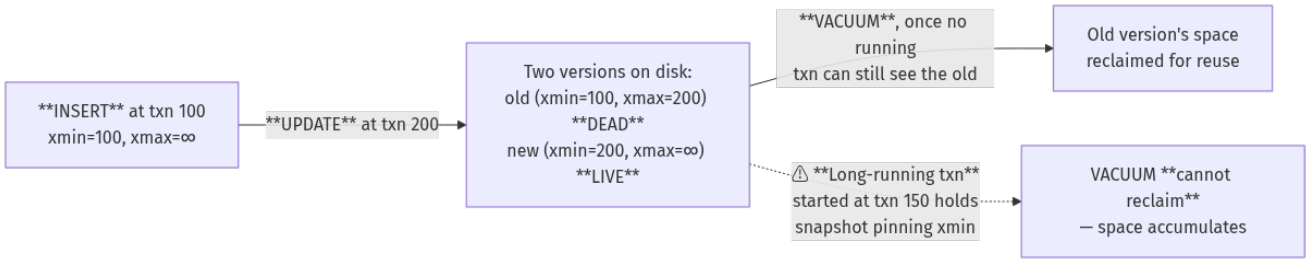


Figure 4: Diagram 4

The dotted path is the operational trap. The single mitigation in practice is `idle_in_transaction_session_timeou` — set to `5min` and Postgres auto-terminates idle sessions older than that.

xid wraparound and freezing

xids are **32-bit** — wrap after ~4 billion txns. To make this safe, Postgres treats half the space as "past" and half as "future" relative to any given xid. Wraparound becomes a problem when an old xid (still stamped on stale rows) ages past the halfway mark and would suddenly appear "in the future."

VACUUM prevents this by **freezing** old rows — rewriting `t_xmin` to `FrozenTransactionId` (value 2). Frozen rows are immune to wraparound forever.

Per-table tracking: `pg_class.relfrozenxid` is the oldest unfrozen xid on this table. The escalation ladder:

`age(relfrozenxid)`	What happens
`> vacuum_freeze_table_age` (150M)	**Aggressive** vacuum on next autovacuum — visits every page (skips the all-visible optimization).

age(relfrozenxid)	What happens
> autovacuum_freeze_max_age (200M)	**Anti-wraparound** vacuum — non-throttled, runs flat-out regardless of time of day. **This is the alert.**
> 2 billion	Postgres **shuts down** to prevent data loss. Single-user mode + VACUUM FREEZE your way out.

The math pgops surfaces:

```
SELECT relname, age(relfrozenxid) AS xid_age,
       round(100.0 * age(relfrozenxid) /
             current_setting('autovacuum_freeze_max_age')::int, 1)
         AS pct_of_freeze_max
FROM pg_class WHERE relkind='r'
ORDER BY age(relfrozenxid) DESC LIMIT 10;
```

Vacuum — what it actually does

A normal VACUUM walks the heap (skipping all-visible pages courtesy of the visibility map), finds dead tuples whose xmax is committed and older than any active snapshot, walks each index removing pointers to those tuples, reclaims the heap line pointers, updates the FSM (free space map) and VM (visibility map), updates relfrozenxid if anything got frozen along the way.

Space goes back to the table, not the OS. For OS-level reclaim use pg_repack (online) — *not* VACUUM FULL (which rewrites under AccessExclusive — multi-minute outage on big tables).

Autovacuum scheduling

Launcher wakes every autovacuum_naptime (1 min default), decides per table:

- **Needs vacuum** if n_dead_tup > autovacuum_vacuum_threshold + autovacuum_vacuum_scale_factor × n_live_tup. Defaults: 50 rows + 20%.
- **Needs analyze** if n_mod_since_analyze > autovacuum_analyze_threshold + autovacuum_analyze_scale_factor × n_live_tup. Defaults: 50 rows + 10%.
- **Needs freeze** if age(relfrozenxid) > autovacuum_freeze_max_age.

Cost-based throttling: each work cycle accumulates cost (1 / page hit, 10 / page read, 20 / page dirtied); sleeps autovacuum_vacuum_cost_delay (default 2ms) when reaching autovacuum_vacuum_cost_limit (200). Effective rate ≈ 800 MB/s dirty pages on defaults. Anti-wraparound vacuum **ignores throttling entirely** — by design.

Views you query for this layer

pg_stat_user_tables — per-table mutation and vacuum stats. The columns that matter:

- n_live_tup, n_dead_tup — live and dead tuple **estimates** (not exact — drift over time).
- n_mod_since_analyze — drives ANALYZE triggers.
- last_vacuum, last_autovacuum, last_analyze, last_autoanalyze — timestamps.
- vacuum_count, autovacuum_count, analyze_count, autoanalyze_count — cumulative.
- n_tup_hot_upd / n_tup_upd — HOT update ratio. Low on an UPDATE-heavy table → consider fillfactor reduction.

Gotchas: counts are estimates (the truth is pgstattuple); partitioned tables get one row per partition not parent; TOAST tables don't appear here — join pg_class.reltoastrelid to find them.

pg_class.relfrozenxid — per-table oldest unfrozen xid. The wraparound horizon (Ch B.3 ladder). Join pg_class.relname and compute age(relfrozenxid) for the "how close to anti-wraparound" report.

pg_stat_progress_vacuum — live progress of any in-flight VACUUM. Columns: pid, relid, phase (initializing / scanning heap / vacuuming indexes / vacuuming heap / cleaning up indexes / truncating heap / performing final cleanup), heap_blks_total / heap_blks_scanned / heap_blks_vacuumed, index_vacuum_count. Sibling views exist for ANALYZE, CREATE INDEX, CLUSTER, base_backup, COPY.

pg_stat_activity with backend_xmin IS NOT NULL — every session pinning the xmin horizon. Order by backend_xmin to find the oldest. That's almost always your bloat villain.

What you intervene with

`pg_terminate_backend()` on the idle-in-txn holder, then `VACUUM (VERBOSE, ANALYZE) <table>;` to catch up. `idle_in_transaction_session_timeout='5min'` as prevention (cluster-wide GUC). `pg_repack` for online bloat reclamation that returns disk to OS. Per-table `ALTER TABLE foo SET (autovacuum_vacuum_scale_factor=0.01, autovacuum_analyze_scale_factor=0.005)` for very large tables where 20% dead tuples is too much to tolerate. For wraparound emergency: nothing other than waiting for the anti-wraparound vacuum to finish (it's already running flat-out). See lab 11.1.D (bloat recovery) and 11.2.B (page-level MVCC + freeze).

B.4 The Locks

DBA loop step: Intervene. **Workflow trigger:** Writes timing out; a migration is "stuck"; PagerDuty says "deadlock_detected" in the logs. **Mental model:** Three lock layers — *heavyweight* table-level (in `pg_locks`), *row-level* (in the tuple's xmax), *predicate* (only under SERIALIZABLE). Most incidents are heavyweight or row-level. `AccessExclusive` blocks **everything**, including SELECT. DDL is just locks acquired by `ALTER TABLE`. **Query to see it:** `SELECT pid, pg_blocking_pids(pid), … FROM pg_stat_activity WHERE wait_event_type = 'Lock';`

The conflict matrix — heatmap-style

The 8 table-level lock modes, with the question "if I want X and they hold Y, do I wait?" Read each row as "I want this mode; the columns show what holders block me." **X** = blocked, · = compatible.

Want ↓ / Hold →	AS	RS	RE	SUE	S	SRE	E	AE
AccessShare (SELECT)	·	·	·	·	·	·	·	**X**
RowShare (SELECT … FOR …)	·	·	·	·	·	·	**X**	**X**
RowExclusive (INSERT/UPDATE/DELETE)	·	·	·	·	**X**	**X**	**X**	**X**
ShareUpdateExclusive (VACUUM, CIC)	·	·	·	**X**	**X**	**X**	**X**	**X**
Share (CREATE INDEX)	·	·	**X**	**X**	·	**X**	**X**	**X**
ShareRowExclusive (CREATE TRIGGER)	·	·	**X**	**X**	**X**	**X**	**X**	**X**
Exclusive (rare; REFRESH MV)	·	**X**	**X**	**X**	**X**	**X**	**X**	**X**
AccessExclusive (ALTER, DROP, TRUNCATE)	**X**	**X**	**X**	**X**	**X**	**X**	**X**	**X**

The single internalize-this fact: AccessExclusive's row is all X. A migration taking AccessExclusive on a hot table blocks every read and write for the duration of the statement.

Row, advisory, predicate locks

Row locks are stored *in the tuple itself* (xmax + infomask bits), not in `pg_locks`. There'd be millions of entries otherwise. To wait on a row lock, your session heavyweight-waits on the *xid* of the holder — surfacing in `pg_locks` as `locktype='transactionid'`, not as a row-level entry. `pg_blocking_pids()` walks this transparently.

Advisory locks are app-defined names; nothing Postgres does conflicts with them. Useful for cron-style "only one of us runs this." Visible as `locktype='advisory'`.

Predicate locks exist only under SERIALIZABLE. The cycle detector uses them to find read-write dependencies between concurrent txns and aborts one if they'd produce a non-serializable schedule. Visible as `mode='SIReadLock'` — never block anything.

Deadlock detection

Postgres doesn't prevent deadlocks — it detects them after `deadlock_timeout` (default 1s). A waiter walks the wait-for graph; if it finds a cycle including itself, it aborts with `deadlock_detected`. The 1s default is because the cycle walk is expensive; most waits resolve in ms and pay zero detection cost.

DDL = locks acquired by ALTER TABLE

The two facts that govern migration safety:

1. **AccessExclusive ≠ table rewrite.** Most AccessExclusive DDLs (catalog-only ADD/DROP/RENAME column) complete in ms.
2. **Table rewrite ≠ AccessExclusive duration.** A rewrite of a 100GB table holds the lock for the whole rewrite (minutes-to-hours).

Statement	Lock	Rewrites?
`ADD COLUMN x int`	AccessExclusive	No
`ADD COLUMN x int NOT NULL DEFAULT 0`	AccessExclusive	**No** since PG 11 [2]; **yes** on PG ≤10

Statement	Lock	Rewrites?
`ADD COLUMN x int NOT NULL DEFAULT now()`	AccessExclusive	**Yes** (volatile default forces rewrite even in PG 11+)
`ALTER COLUMN x TYPE bigint`	AccessExclusive	Yes
`ALTER COLUMN x SET NOT NULL`	AccessExclusive	No, but full scan to verify
`ADD CONSTRAINT chk … NOT VALID` then `VALIDATE`	AccessExcl then SUE	No, no rewrite — split-validation is the safe pattern
`DROP COLUMN x`	AccessExclusive	No (catalog-only)
`CREATE INDEX`	Share	No, but blocks writes for build
`CREATE INDEX CONCURRENTLY`	ShareUpdateExclusive	No

The **fast-default exception** (PG 11+): the default value is stored in `pg_attribute.attmissingval` and served on read for old rows; only new/updated rows write physically [3]. The historically dangerous pattern is only dangerous on PG ≤10 or with volatile defaults.

The safe DDL shape — expand → backfill → contract

For *any* schema change otherwise needing long AccessExclusive:

1. **Expand.** Add the new shape, nullable/unvalidated/non-concurrent. Weak lock, instant.
2. **Backfill.** Populate in small batches with row-level locks only — `LIMIT 5000 FOR UPDATE SKIP LOCKED`, explicit COMMIT after each batch, `pg_sleep(0.1)` between.
3. **Contract.** Tighten constraints (SET NOT NULL, validate FK). Cheap because data is already correct.

Guardrails

```
SET lock_timeout = '2s';
SET statement_timeout = '30s';
ALTER TABLE …;
```

`lock_timeout` errors out if the lock can't be acquired in time — migration aborts cleanly. `statement_timeout` backstops "this should be a fast catalog change." Every migration tool should set both per migration.

Views you query for this layer

`pg_locks` — one row per held or requested heavyweight lock. Columns that matter:

- `locktype` — `relation` (table/index), `transactionid` (xid waits — row-level waits surface here), `virtualxid` (session's local txn id), `tuple`, `page`, `advisory`, `extend`, `object`.
- `mode` — one of the 8 modes from the heatmap above.
- `granted` — `false` = waiting; `true` = held.
- `relation` — OID, castable to `regclass` for table name (null for transactionid locks).
- `pid` — backend pid, the join key to pg_stat_activity.

Gotcha — row-level lock waits don't appear as `locktype='tuple'`. They appear as `locktype='transactionid'` (the waiter is heavyweight-waiting on the *xid* of the holder), with the relation column null. `pg_blocking_pids()` walks this transparently — use it rather than reading pg_locks raw for lock-chain queries.

`pg_stat_activity` (already covered in B.3) — joined to pg_locks via `pid` for query / wait_event / state context. `wait_event_type='Lock'` filters for sessions waiting on a lock.

`pg_blocking_pids(pid)` — function returning array of pids whose locks are blocking the given pid. Accounts for both heavyweight lock waits AND row-level (xid) waits. This is the function the canonical lock-chain query uses; pgops's chain-building does a recursive walk of it to find the root blocker.

What you intervene with

`pg_cancel_backend(pid)` to cancel just the current query in a session (gentle); `pg_terminate_backend(pid)` to kill the session entirely (forces transaction rollback, releases all locks). Use cancel first if there's any chance the session might recover; terminate when the session is the problem itself (idle in txn, blocking writers).

`lock_timeout` + `statement_timeout` set before every migration (and ideally cluster-wide for ALTER-running roles) to make a stuck migration error out instead of hanging.

`pgops review` (libpg_query-based AST analysis) to flag dangerous DDL patterns before they reach prod:

Pattern	Risk	Recommended rewrite
ADD COLUMN NOT NULL with non-constant default	Rewrite under AccessExclusive	Expand → backfill → contract
ALTER COLUMN TYPE	Always rewrites	New column + backfill + swap
CREATE INDEX without CONCURRENTLY on >threshold table	Blocks writes	Use CONCURRENTLY
ADD FOREIGN KEY without NOT VALID	Full scan under AccessExclusive	NOT VALID + later VALIDATE (SUE)
CREATE INDEX CONCURRENTLY inside a transaction	Postgres rejects	Split CONCURRENTLY into its own migration

See lab 11.1.C (lock chain triage) and 11.1.E (migration safety race).

Part C — The lab

Two scenarios. **11.1** is a single on-call shift against one seeded e-commerce DB — six incidents, six labs. **11.2** is three mechanism-reveal sandboxes against purpose-built tiny side tables. Each lab opens with where it sits in the frame.

11.0 Shared setup

```
docker run -d --name pgops-lab -p 5432:5432 \
  -e POSTGRES_PASSWORD=postgres \
  postgres:18 \
  -c shared_preload_libraries=pg_stat_statements \
  -c pg_stat_statements.track=all \
  -c wal_level=logical \
  -c max_replication_slots=10 \
  -c max_wal_senders=10 \
  -c idle_in_transaction_session_timeout=0
```

Install pg_partman (only used in 11.2.C):

```
docker exec pgops-lab bash -c \
  "apt-get update && apt-get install -y postgresql-18-partman"
```

Connect: `psql postgresql://postgres:postgres@localhost:5432/postgres`. Then create the schema, seed, enable extensions (~3 min):

```
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

CREATE TABLE customers (id bigserial PRIMARY KEY, email text NOT NULL,
                        signup_at timestamptz NOT NULL DEFAULT now());
CREATE TABLE orders (id bigserial PRIMARY KEY,
                     customer_id bigint NOT NULL REFERENCES customers(id),
                     total_cents int NOT NULL, status text NOT NULL,
                     created_at timestamptz NOT NULL);
CREATE TABLE sessions (id bigserial PRIMARY KEY, user_id bigint NOT NULL,
                       token text NOT NULL, last_seen timestamptz NOT NULL);

INSERT INTO customers (email) SELECT 'user'||g||'@example.com'
  FROM generate_series(1, 100000) g;
INSERT INTO orders (customer_id, total_cents, status, created_at)
  SELECT (random()*99999+1)::bigint, (random()*100000)::int,
         (ARRAY['pending','shipped','delivered','cancelled'])[1+(random()*3)::int],
         now() - (random() * interval '365 days')
  FROM generate_series(1, 5000000);
INSERT INTO sessions (user_id, token, last_seen)
  SELECT (random()*99999+1)::bigint, md5(random()::text),
         now() - (random() * interval '7 days')
  FROM generate_series(1, 1000000);

ANALYZE;
```

11.1 Scenario 1 — on-call shift against the e-commerce DB

11.1.A — Planner observation

> **Loop step:** Diagnose · **Layer:** B.1 Planner · **You hit this when:** investigating perf to understand what the planner is choosing.

```
SET work_mem = '64kB';
EXPLAIN (ANALYZE, BUFFERS) SELECT c.id, count(o.id) FROM customers c
  LEFT JOIN orders o ON o.customer_id = c.id GROUP BY c.id;
```

Key line in output:

```
 HashAggregate  ...  Batches: 33  Memory Usage: 73kB  Disk Usage: 4768kB
```

`Disk Usage: 4768kB` — spilled to temp files. Re-run with `SET work_mem='64MB';` — `Batches: 1`, no disk usage, ~3× faster.

Force a tiny outer side and watch Nested Loop win:

```
EXPLAIN ANALYZE SELECT * FROM customers c JOIN orders o
  ON o.customer_id = c.id WHERE c.id IN (1,2,3);
```

```
 Nested Loop  (actual rows=~150)
   ->  Index Scan using customers_pkey on customers c
   ->  Index Scan using orders_customer_id_idx on orders o
```

Cold vs warm cache:

```
DISCARD ALL;
EXPLAIN (ANALYZE, BUFFERS) SELECT count(*) FROM orders WHERE status='shipped';
-- First run:  Buffers: shared hit=312 read=37291
-- Re-run:     Buffers: shared hit=37603 read=0
```

`hit` dominates on re-run — table now in `shared_buffers`. **Teardown:** `RESET work_mem;`

11.1.B — Slow query → index, with and without CONCURRENTLY

Loop step: Diagnose + Intervene · **Layer:** B.1 Planner · **You hit this when:** pg_stat_statements shows a query burning total time and EXPLAIN shows Seq Scan.

Generate the slow query:

```
SELECT pg_stat_statements_reset();
DO $$ BEGIN FOR i IN 1..100 LOOP
  PERFORM count(*) FROM orders WHERE created_at > now() - interval '7 days';
END LOOP; END $$;
```

Find it:

```
SELECT substring(query,1,40) AS q, calls,
       round(total_exec_time::numeric, 1) AS total_ms
FROM pg_stat_statements WHERE query LIKE '%FROM orders%'
ORDER BY total_exec_time DESC LIMIT 1;
```

```
                    q                     | calls | total_ms
------------------------------------------+-------+----------
 SELECT count(*) FROM orders WHERE creat |   100 |  12842.3
```

Naive blocking build. **[Tab B]** keep a writer running:

```
DO $$ BEGIN FOR i IN 1..1000 LOOP
  UPDATE orders SET status='pending' WHERE id = (random()*5e6)::bigint;
  PERFORM pg_sleep(0.05);
END LOOP; END $$;
```

[Tab A] the blocking build:

```
CREATE INDEX orders_created_at_idx ON orders (created_at);
```

[Tab C] observe — Tab B is Lock-waiting:

```
SELECT pid, wait_event_type, wait_event, substring(query,1,30) AS q
FROM pg_stat_activity WHERE state='active' AND pid != pg_backend_pid();
```

```
  pid  | wait_event_type | wait_event |       q
-------+-----------------+------------+----------------
 24512 | (null)          | (null)     | CREATE INDEX o
 24588 | Lock            | relation   | UPDATE orders
```

Redo with CONCURRENTLY:

```
DROP INDEX orders_created_at_idx;
CREATE INDEX CONCURRENTLY orders_created_at_idx ON orders (created_at);
```

Tab B sees no Lock waits — writes flow throughout. **Teardown:** `DROP INDEX orders_created_at_idx;` `SELECT pg_stat_statements_reset();`

11.1.C — Lock chain triage

Loop step: Diagnose + Intervene · **Layer:** B.4 Locks · **You hit this when:** writes are timing out, pg_stat_activity shows Lock waits.

[Tab A] take row lock, go idle:

```
BEGIN;
SELECT * FROM orders WHERE id IN (1,2,3) FOR UPDATE;
-- (leave open)
```

[Tab B], **[Tab C]**: try to update locked rows (will block):

```
-- Tab B
UPDATE orders SET status='cancelled' WHERE id = 3;
-- Tab C
UPDATE orders SET status='shipped'   WHERE id = 4;
```

[Tab D] observe the chain (vertical mode for readability):

```
\x on
SELECT blocked.pid AS blocked_pid, blocking.pid AS blocking_pid,
       blocking.state AS blocking_state,
       age(now(), blocked.xact_start) AS blocked_for,
       substring(blocking.query, 1, 40) AS blocking_q
FROM pg_stat_activity blocked
JOIN pg_stat_activity blocking
  ON blocking.pid = ANY(pg_blocking_pids(blocked.pid))
WHERE blocked.wait_event_type = 'Lock';
\x off
```

```
-[ RECORD 1 ]--+---------------------------------------
blocked_pid    | 24911
blocking_pid   | 24820
blocking_state | idle in transaction
blocked_for    | 00:00:42
blocking_q     | SELECT * FROM orders WHERE id IN (1,2,3
```

Pid 24820 is the root — idle for 42s, blocking two writers. Kill it:

```
SELECT pg_terminate_backend(24820);
```

Tab B/C unblock. **Teardown:** already done by terminate.

11.1.D — Autovacuum bloat recovery

Loop step: Diagnose + Intervene · **Layer:** B.3 MVCC + Vacuum · **You hit this when:** pg_stat_user_tables shows a high dead_pct and the table won't shrink.

[Tab A] pin the xmin horizon:

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT 1;  -- assigns snapshot
-- (leave open)
```

[Tab B] generate dead tuples:

```
UPDATE sessions SET last_seen = now() WHERE id IN
  (SELECT id FROM sessions LIMIT 500000);
UPDATE sessions SET last_seen = now() WHERE id IN
  (SELECT id FROM sessions LIMIT 500000);
```

Observe:

```
SELECT relname, n_live_tup, n_dead_tup,
       round(100.0*n_dead_tup/nullif(n_live_tup+n_dead_tup, 0), 1) AS dead_pct
FROM pg_stat_user_tables WHERE relname='sessions';
```

```
 relname  | n_live_tup | n_dead_tup | dead_pct
----------+------------+------------+----------
 sessions |    1000000 |    1000000 |     50.0
```

`VACUUM (VERBOSE) sessions;` will print:

```
INFO:  ... 1000000 are dead but not yet removable
```

"Not yet removable" — Tab A pins the horizon. Find the holder:

```
SELECT pid, age(now(), xact_start) AS txn_age, backend_xmin
FROM pg_stat_activity WHERE backend_xmin IS NOT NULL
ORDER BY backend_xmin LIMIT 1;
```

```
  pid  | txn_age  | backend_xmin
-------+----------+--------------
 26204 | 00:04:21 |       418293
```

Kill it, vacuum, **and watch progress live**. Re-generate dead tuples and force a vacuum in Tab A:

```
-- Tab A
SELECT pg_terminate_backend(26204);
UPDATE sessions SET last_seen = now() WHERE id IN
  (SELECT id FROM sessions LIMIT 300000);
VACUUM (VERBOSE, ANALYZE) sessions;
```

[Tab B] poll repeatedly during the VACUUM:

```
\x on
SELECT pid, relid::regclass, phase,
       heap_blks_total, heap_blks_scanned, heap_blks_vacuumed,
       index_vacuum_count
FROM pg_stat_progress_vacuum;
\x off
```

```
-[ RECORD 1 ]------+---------
pid                | 26780
relid              | sessions
phase              | scanning heap
heap_blks_total    | 12345
heap_blks_scanned  | 4821
heap_blks_vacuumed | 0
index_vacuum_count | 0
```

Watch `phase` flip to `vacuuming indexes` → `vacuuming heap` → `cleaning up indexes`. **Teardown:** the VACUUM is the cleanup.

11.1.E — Migration safety (the AccessExclusive race)

> **Loop step:** Intervene (avoiding incident) · **Layer:** B.4 Locks · **You hit this when:** deploying any non-trivial migration on a hot table.

To force a rewrite on modern PG 18 (fast-default makes the historic constant-default pattern safe), use a volatile default:

[Tab B] start a workload:

```
DO $$ BEGIN FOR i IN 1..200 LOOP
  PERFORM count(*) FROM orders WHERE id = (random()*5e6)::bigint;
  PERFORM pg_sleep(0.1);
END LOOP; END $$;
```

[Tab A] the dangerous DDL (forces full rewrite because `random()` is volatile):

```
ALTER TABLE orders ADD COLUMN risk_score float NOT NULL DEFAULT random();
```

[Tab C] observe the queue:

```
SELECT locktype, mode, granted, pid FROM pg_locks
WHERE relation = 'orders'::regclass ORDER BY granted DESC;
```

```
 locktype |        mode         | granted |  pid
----------+---------------------+---------+-------
 relation | AccessExclusiveLock | t       | 26901
 relation | AccessShareLock     | f       | 27012
```

The AccessExclusive blocks even Tab B's SELECT. Tab A holds the lock for the entire rewrite (~1 min on 5M rows).

When Tab A finishes, redo as expand → backfill → contract:

```
ALTER TABLE orders DROP COLUMN risk_score;
-- expand (instant; fast-default applies because default is constant)
ALTER TABLE orders ADD COLUMN risk_score int;
-- backfill (row-level locks only, no AccessExclusive)
DO $$ DECLARE rows_done int := 1;
BEGIN WHILE rows_done > 0 LOOP
  WITH batch AS (SELECT id FROM orders WHERE risk_score IS NULL
                 LIMIT 50000 FOR UPDATE SKIP LOCKED)
  UPDATE orders SET risk_score = (random()*100)::int
  WHERE id IN (SELECT id FROM batch);
  GET DIAGNOSTICS rows_done = ROW_COUNT;
  COMMIT;
  PERFORM pg_sleep(0.05);
END LOOP; END $$;
-- contract (brief AccessExclusive, scan finds no nulls, no rewrite)
ALTER TABLE orders ALTER COLUMN risk_score SET DEFAULT 0;
ALTER TABLE orders ALTER COLUMN risk_score SET NOT NULL;
```

Tab C's `pg_locks` during backfill shows only `RowExclusiveLock` — no Tab B waits. **Teardown:** `ALTER TABLE orders DROP COLUMN risk_score;`

11.1.F — Logical decoding + orphan slot

Loop step: Observe + Intervene · **Layer:** B.2 WAL · **You hit this when:** disk filling, replication lag alerts, or an orphan slot defined but unused.

Create publication + slot:

```
CREATE PUBLICATION orders_pub FOR TABLE orders;
SELECT * FROM pg_create_logical_replication_slot('orders_slot', 'test_decoding');
```

Generate events and pull decoded:

```
INSERT INTO orders (customer_id, total_cents, status, created_at)
VALUES (1, 1000, 'pending', now());
SELECT lsn, xid, substring(data, 1, 60) AS data
FROM pg_logical_slot_get_changes('orders_slot', NULL, NULL);
```

```
    lsn     |  xid   |                        data
------------+--------+-------------------------------------
 0/3A0192A0 | 41832  | BEGIN 41832
 0/3A019388 | 41832  | table public.orders: INSERT: id[bigi
 0/3A019410 | 41832  | COMMIT 41832
```

That's WAL re-projected as row-level events. `get_changes` *consumes*; `peek_changes` doesn't.

Stage orphan-slot — stop consuming, generate WAL:

```
INSERT INTO orders (customer_id, total_cents, status, created_at)
SELECT (random()*99999)::bigint, (random()*100000)::int, 'pending', now()
FROM generate_series(1, 100000);
```

Detect:

```
SELECT slot_name, active,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn))
         AS retained_wal
FROM pg_replication_slots;
```

```
  slot_name   | active | retained_wal
--------------+--------+--------------
 orders_slot  | f      | 24 MB
```

`active=f` + non-trivial `retained_wal` = the orphan pattern. **Teardown:**

```
SELECT pg_drop_replication_slot('orders_slot');
DROP PUBLICATION orders_pub;
```

11.2 Scenario 2 — standalone mechanism-reveal sandboxes

11.2.A — Index type comparison

Loop step: Diagnose (pre-deploy: which index to use?) · **Layer:** B.1 Planner · **Sandbox table.**

```
CREATE TABLE idx_demo (id bigserial PRIMARY KEY, category text, ts timestamptz,
                       status text NOT NULL, description text NOT NULL);
INSERT INTO idx_demo (category, ts, status, description)
SELECT (ARRAY['a','b','c'])[1+(random()*2)::int],
       now() - (random() * interval '365 days'),
       CASE WHEN random() < 0.05 THEN 'active' ELSE 'archived' END,
       md5(random()::text)||' '||md5(random()::text)
FROM generate_series(1, 1000000);
ANALYZE idx_demo;

CREATE INDEX idx_demo_ts_btree ON idx_demo (ts);
CREATE INDEX idx_demo_ts_brin  ON idx_demo USING BRIN (ts);
CREATE INDEX idx_demo_status_partial ON idx_demo (id) WHERE status = 'active';
CREATE INDEX idx_demo_descr_gin ON idx_demo
  USING GIN (to_tsvector('english', description));

SELECT indexrelname,
       pg_size_pretty(pg_relation_size(indexrelid)) AS size
FROM pg_stat_user_indexes WHERE relname='idx_demo'
ORDER BY pg_relation_size(indexrelid) DESC;
```

```
       indexrelname        |  size
---------------------------+----------
 idx_demo_descr_gin        | 28 MB
 idx_demo_ts_btree         | 21 MB
 idx_demo_status_partial   | 1080 kB
 idx_demo_ts_brin          |   48 kB
```

BRIN ~430× smaller than B-tree on the same column. Partial ~20× smaller than full B-tree because only 5% of rows match. GIN is bigger but is the *only* index that handles the `@@` operator. **Teardown:** `DROP TABLE idx_demo;`

11.2.B — MVCC at page level

Loop step: Diagnose (deep / occasional) · **Layer:** B.3 MVCC + Vacuum · **Sandbox table.**

```
CREATE EXTENSION IF NOT EXISTS pageinspect;
CREATE TABLE mvcc_demo (id int PRIMARY KEY, name text);
INSERT INTO mvcc_demo VALUES (1,'alice'),(2,'bob'),(3,'carol');

SELECT lp, t_xmin, t_xmax, t_ctid
FROM heap_page_items(get_raw_page('mvcc_demo', 0));
```

```
 lp | t_xmin | t_xmax | t_ctid
----+--------+--------+--------
  1 | 421902 |      0 | (0,1)
  2 | 421902 |      0 | (0,2)
  3 | 421902 |      0 | (0,3)
```

UPDATE one row:

```
UPDATE mvcc_demo SET name = 'ALICE' WHERE id = 1;
SELECT lp, t_xmin, t_xmax, t_ctid
FROM heap_page_items(get_raw_page('mvcc_demo', 0));
```

```
 lp | t_xmin | t_xmax | t_ctid
----+--------+--------+--------
  1 | 421902 | 421905 | (0,4)   <-- old, dead, points to new
  2 | 421902 |      0 | (0,2)
  3 | 421902 |      0 | (0,3)
  4 | 421905 |      0 | (0,4)   <-- new, fresh xmin
```

VACUUM, then VACUUM FREEZE, observe `t_xmin = 2` (`FrozenTransactionId`):

```
VACUUM mvcc_demo;
VACUUM FREEZE mvcc_demo;
SELECT lp, t_xmin, t_xmax, (t_xmin = '2'::xid) AS frozen
FROM heap_page_items(get_raw_page('mvcc_demo', 0))
WHERE lp_flags = 1;
```

```
 lp | t_xmin | t_xmax | frozen
----+--------+--------+--------
  2 |      2 |      0 | t
  3 |      2 |      0 | t
  4 |      2 |      0 | t
```

`t_xmin=2` is the magic value. These rows are now immune to wraparound forever. **Teardown:** `DROP TABLE mvcc_demo;`

11.2.C — Partition rotation with pg_partman

Loop step: Intervene (scheduled maintenance) · **Layer:** B.4 Locks (DDL semantics) + cross-cutting partitioning · **Sandbox table.**

```
CREATE EXTENSION IF NOT EXISTS pg_partman;

CREATE TABLE events_partman (id bigserial, event_type text NOT NULL,
                             created_at timestamptz NOT NULL)
  PARTITION BY RANGE (created_at);

SELECT partman.create_parent(
  p_parent_table   := 'public.events_partman',
  p_control        := 'created_at',
  p_type           := 'native',
  p_interval       := '1 day',
  p_premake        := 7,
  p_start_partition := (now() - interval '3 days')::date::text);

UPDATE partman.part_config SET retention = '30 days'
WHERE parent_table = 'public.events_partman';
```

partman creates 11 partitions (3 past + today + 7 future). Insert routes automatically:

```
INSERT INTO events_partman (event_type, created_at)
VALUES ('signup', now() - interval '2 days'),
       ('view',   now() + interval '5 days');
SELECT tableoid::regclass AS partition, event_type
FROM events_partman ORDER BY created_at;
```

```
        partition         | event_type
--------------------------+------------
 events_partman_p20260621 | signup
 events_partman_p20260628 | view
```

Maintenance — create future, drop expired:

```
SELECT partman.run_maintenance(p_analyze := false);
```

Teardown:

```
SELECT partman.undo_partition(p_parent_table := 'public.events_partman',
                              p_target_table := 'public.events_partman');
DROP TABLE events_partman;
DELETE FROM partman.part_config WHERE parent_table = 'public.events_partman';
```

Part D — Quick reference

Term	Meaning
ACID	Atomicity / Consistency / Isolation / Durability — the four transaction guarantees.
AccessExclusive	Strongest table lock. Held by most ALTER, DROP, TRUNCATE. Blocks everything including SELECT.
advisory lock	App-defined lock; nothing Postgres does conflicts with it. Cross-session coordination.
ANALYZE	Refresh `pg_statistic` column histograms/MCVs. Cheap, frequent, fixes most bad plans.
autovacuum	Background workers running VACUUM + ANALYZE on tunable thresholds.
backend	Postgres process serving one client connection.
B-tree	Balanced-tree index, default. Equality, range, sort, prefix.
BRIN	Block Range Index. Summary per page-range. Tiny, fast on physically-ordered data.
buffer cache	`shared_buffers` — Postgres's in-memory page cache, shared across backends.
catalog	System tables (`pg_catalog` schema) describing every DB object.
checkpoint	Flush of dirty `shared_buffers` to data files + checkpoint record to WAL.
CONCURRENTLY	Non-blocking variant of CREATE INDEX / DROP INDEX / REINDEX.
dead tuple	Row version superseded by UPDATE/DELETE. Reclaimed by VACUUM.
deadlock_timeout	How long to wait before scanning for a deadlock cycle. Default 1s.
EXPLAIN	Show planner's chosen plan. ANALYZE runs it; BUFFERS adds cache stats.
fast-path lock	Heavyweight lock in per-backend memory instead of shared table. Cuts contention.
FrozenTransactionId	xid value `2` — marks tuples that have been frozen.
FSM	Free Space Map. Per-table bitmap of approx free space per page.
GIN	Generalized Inverted Index. Full-text, JSONB containment, arrays.
GiST	Generalized Search Tree. Contains/overlaps/nearest (PostGIS, ranges, KNN).
HOT update	Heap-Only Tuple update. New version on same page, no index touched.
idle in transaction	Session that ran BEGIN, did work, went idle. Pins the MVCC horizon.
LSN	Log Sequence Number. 64-bit byte offset into WAL, hex like `0/16B5A88`.
libpg_query	Postgres's parser as a standalone library. Used by `pgops review`.
MVCC	Multi-Version Concurrency Control. UPDATE = new version + dead old; readers snapshot.
page	8 KB unit of storage and I/O.
pageinspect	Extension exposing raw page contents as SQL functions.
pg_partman	Partition-management extension. Premake + retention automation.
pg_stat_statements	Extension aggregating per-normalized-query timing across all calls.
pgstattuple	Extension for exact dead-tuple/live-tuple/free byte counts per table.
postmaster	Parent Postgres process. Forks backends, supervises workers.
predicate lock (SIRead)	SERIALIZABLE-only. Records reads for cycle detection. Doesn't block.

Term	Meaning
relfrozenxid	Per-table: oldest unfrozen xid. Drives anti-wraparound triggers.
replication slot	Per-subscriber bookkeeping. Pins WAL until subscriber confirms.
rewriter	Query-processing stage that expands views, applies rules + RLS.
RowExclusive	Lock taken by INSERT/UPDATE/DELETE. Compatible with itself.
Seq Scan	Sequential scan of every page in a table.
snapshot	Set of row versions visible to a transaction.
SSI	Serializable Snapshot Isolation. Postgres's SERIALIZABLE implementation.
ShareUpdateExclusive	Lock taken by VACUUM, ANALYZE, CREATE INDEX CONCURRENTLY. Doesn't block reads/writes.
TOAST	The Oversized-Attribute Storage Technique. Values > ~2 KB go to a sidecar table.
VACUUM	Reclaims dead tuples, updates FSM/VM, freezes old xids. Non-blocking.
VACUUM FULL	Rewrites table under AccessExclusive. Almost never right in production.
visibility map (VM)	Per-page bits: all-visible (enables index-only scans), all-frozen.
WAL	Write-Ahead Log. Append-only record of every change. Source of truth.
wal_level	`minimal`, `replica` (default), `logical` (enables logical decoding).
work_mem	Per-operation memory budget for sort/hash. Per op, not per query.
xid	32-bit transaction id, stamped on row's xmin/xmax.
xid wraparound	xids wrap at 2^32; VACUUM freezes old rows to prevent.
xmin / xmax	Hidden columns on every row version: creator xid / invalidator xid.

Sources

1. https://www.postgresql.org/about/news/postgresql-184-1710-1614-1518-and-1423-released-3297/
2. https://www.crunchydata.com/blog/when-does-alter-table-require-a-rewrite
3. https://levelup.gitconnected.com/what-really-happens-when-you-add-a-default-column-in-postgresql-6295e00e8606
4. https://learnomate.org/why-every-dba-should-master-pg-stat-activity/
5. https://full-stack.blog/blog/18_inside_postgresql/
6. https://www.interdb.jp/pg/
7. https://www.db24x7.com/guides/postgresql-monitoring-pg-stat/
8. https://www.koenig-solutions.com/blog/postgresql-dba-roles-and-responsibilities-skills-jobs-description-salary